UW Bothell Campus Pulse — Project Proposal & Technical Specification

UW Community Impact Statement

Students at UW Bothell frequently miss critical campus information — club events, guest lectures, workshops, and emergency alerts — because email is an unreliable delivery channel that many students deprioritize. We will build a browser-based interactive map of UW Bothell that automatically pulls event listings and emergency alerts from UW's official sources in real time. Built on OpenStreetMap, the map displays color-coded pins on relevant buildings whenever something happens on campus. Users can opt into email digests or push notifications, but the core experience requires no account — just opening a link.

AI Integration Strategy

Strategy 1 — NLP Alert Summarizer Raw UW Alert RSS messages are unstructured text (e.g. "Police activity near UW1, avoid the area"). Every time a new RSS item is detected server-side, a call is made to the GPT-4o-mini API with a structured prompt that instructs the model to return a JSON object containing: building name, incident type, severity (low/medium/high), and recommended student action. This JSON directly drives the map rendering — without it, there is no pin location and no structured card. AI is not optional to the feature; it is the feature.

Strategy 2 — Personalized Event Recommender When a student logs in with their UW NetID, user interaction data (events viewed, clicked, saved) is logged in a PostgreSQL database. A collaborative filtering model built with Python's Surprise library generates a ranked "For You" feed of upcoming events personalized to each user. First-time users receive cold-start recommendations seeded by their declared UW major via the UW Student Web Service API, ensuring every student sees relevant events immediately rather than a raw chronological dump.

The model activation threshold is defined at two levels: (1) *globally*, the collaborative filtering model activates once the interaction matrix exceeds 50 records; (2) *per-user*, any logged-in user with fewer than 10 personal interaction records continues to receive popularity-ranked content regardless of global matrix size, preventing the edge case where a returning-but-sparse user receives lower-quality recommendations than a brand-new user. This ensures recommendation quality is monotonically non-decreasing as both global and individual data accumulates.

During the first two weeks of deployment, all logged-in users will receive popularity-ranked feeds until both thresholds are met. All interaction data is stored anonymously and in compliance with UW FERPA guidelines; NetID is hashed before storage.

Reliability & Error Handling

Both AI-dependent systems have defined fallback behaviors to ensure the application remains functional under failure conditions:

NLP Summarizer — API failure or malformed JSON: If GPT-4o-mini returns an error or unparseable response, the system falls back to displaying the raw RSS text in a generic "Alert" pin placed at the campus center coordinates. A "⚠ Live AI summarization unavailable" banner is shown to the user so they are never silently given stale or absent data. Success threshold: the NLP summarizer must return valid, parseable JSON for ≥90% of inputs across a 20-alert test suite before the Week 8 MVP is considered complete.

RSS feed down: The server caches the last successful fetch with a UTC timestamp. The map displays a "Last updated X minutes ago" indicator at all times, making data freshness visible to users — critical for an emergency alert system.

Recommender — UW Student Web Service unavailable: If the major-seeding API call fails during cold start, the system falls back to popularity-ranked events (most viewed campus-wide in the past 7 days) so new users always see a meaningful feed rather than an empty page.

Privacy & Data Governance

NetID authentication is handled entirely through UW's official SSO (Shibboleth), meaning raw credentials never touch our application servers. Our system receives only a hashed user token post-authentication.

The PostgreSQL database is hosted on Supabase (cloud-hosted, access restricted to project team members via role-based access control). We store only three fields per interaction record: hashed NetID, event ID, and interaction timestamp — no names, emails, or personally identifiable information. Interaction logs older than one academic quarter are purged automatically via a scheduled Supabase Edge Function. The project will be reviewed against UW's FERPA compliance guidelines prior to public deployment, and a data handling summary will be included in the project website's technical documentation.

Repository Access

https://github.com/HoneyBadger4334/CSS382_DYOP

Project Website Plan

The public-facing project website will be hosted on Vercel and structured as follows:

Home: Project overview, problem statement, and a live embedded demo of the map.

How It Works: A high-level architecture diagram covering four clearly labeled layers: (1) the RSS ingestion pipeline showing the polling interval and caching logic; (2) the NLP summarizer showing the GPT-4o-mini API call, JSON schema, and fallback path; (3) the recommender system showing the Surprise model, cold-start threshold, per-user activation threshold, and popularity fallback; and (4) the database layer showing the three stored fields, Supabase hosting, and the purge schedule. Accompanying prose descriptions of the full tech stack (OpenStreetMap, Next.js, FastAPI, PostgreSQL/Supabase, GPT-4o-mini, Surprise).

The website design follows a two-column layout with a persistent left-side navigation bar. The Home page features a full-width embedded map demo above the fold. The How It Works page uses a horizontal swimlane diagram for the architecture, with collapsible prose sections beneath each layer. The User Guide uses a numbered step format with annotated screenshots. A Figma wireframe for all four pages will be completed and linked in the repository by end of Week 7, prior to the website draft going live in Week 9, giving the team and instructor early visibility into the design direction.

User Guide: Step-by-step instructions for using the map, opting into notifications, and logging in with NetID for personalized recommendations.

About: Team member names and individual contribution areas.

A draft of the website will be live by the end of Week 9 to allow time for feedback and polish before the final presentation. The final version — including the completed architecture diagram — will be deployed by end of Week 10.

Milestone Roadmap

Milestone	Deliverable	Acceptance Criteria	Internal Deadline
Week 7	Repo live; OpenStreetMap renders UW Bothell; at least one hardcoded test pin visible; Figma wireframe for all four website pages complete and linked in repo	Map loads in browser with correct UW Bothell bounding box; test pin appears at correct coordinates; wireframe accessible via repo link	End of Week 7

Week 8 (MVP)	Live RSS feed pulling UW Alerts; NLP summarizer returning valid JSON; pins rendering on correct buildings; fallback banner functional	NLP summarizer returns valid, parseable JSON for ≥90% of inputs across a 20-alert test suite; fallback banner renders on simulated API failure within 3 seconds	End of Week 8
Week 9 (Testing)	Event recommender live for logged-in users; both global (50-record) and per-user (10-record) activation thresholds enforced; cold-start and popularity fallback working; bug fixes from peer testing session; website draft live on Vercel	≥5 peer testers complete the user flow with no critical bugs reported; ≥3 of 5 testers rate their recommendations as "relevant" or "very relevant" on a post-session survey; cold-start fallback verified with a fresh test account; per-user threshold verified by testing a returning account with <10 interactions against a fresh account; website draft accessible at public URL	End of Week 9
Week 10 (Final)	Public deployment stable; project website finalized with architecture diagram and user guide; all rubric criteria verified; peer review submitted	Zero critical bugs in 24-hour uptime check; all four architecture diagram layers present and labeled; rubric self-assessment completed by each team member	End of Week 10

UW Bothell Campus Pulse — Project Proposal & Technical Specification UW Community Impact Statement Students at UW Bothell frequently miss critical ...

PASTED

Prompt: Given the attached assignment specification, grade the proposal out of 5. Identify 3 strengths and 3 weaknesses of the proposal.

10:45 PM

Looking at the assignment spec and the proposal, here's my assessment:

Grade: 5/5

Strengths

1. AI is genuinely central, not cosmetic. The rubric's hardest criterion is that AI must be "meaningfully embedded" rather than a side chat. This proposal nails it with two distinct integrations: the NLP summarizer where AI *is* the pin-placement logic (no valid JSON = no structured alert), and a collaborative filtering recommender with well-defined activation thresholds. Either would satisfy the requirement; having both, with clear technical rationale, is exceptional.

2. Failure modes are anticipated at every layer. The proposal specifies fallback behaviors for API failure, malformed JSON, RSS outages, and cold-start edge cases — including the subtle returning-but-sparse user problem. M[illegible]tudent proposals ignore error handling entirely. Having a quantified success threshold (≥90% valid JSON across a 20-alert test

Write a message...

Sonnet 4.6

Grade: 5/5

Strengths

1. AI is genuinely central, not cosmetic. The rubric's hardest criterion is that AI must be "meaningfully embedded" rather than a side chat. This proposal nails it with two distinct integrations: the NLP summarizer where AI *is* the pin-placement logic (no valid JSON = no structured alert), and a collaborative filtering recommender with well-defined activation thresholds. Either would satisfy the requirement; having both, with clear technical rationale, is exceptional.

2. Failure modes are anticipated at every layer. The proposal specifies fallback behaviors for API failure, malformed JSON, RSS outages, and cold-start edge cases — including the subtle returning-but-sparse user problem. Most student proposals ignore error handling entirely. Having a quantified success threshold (≥90% valid JSON across a 20-alert test suite) gives the team a concrete, testable acceptance criterion rather than a vague "it works."

3. The milestone table is genuinely evaluable. Each milestone has a specific deliverable, a measurable acceptance criterion, and an internal deadline. Week 9's criteria — ≥5 peer testers, ≥3/5 rating recommendations as relevant, cold-start verified with a fresh account — are specific enough that an instructor can verify them without ambiguity. This directly addresses the rubric's "adherence to proposed timeline" criterion.

Weaknesses

1. No mention of group size or individual contribution areas (beyond the About page placeholder). The assignment requires 2–3 people and heavily weights peer review (15 pts). The proposal never states how many members are on the team or how responsibilities are divided, which is a gap given that equitable work distribution is explicitly evaluated.

2. The OpenStreetMap + building-pin architecture is underspecified. The proposal describes color-coded pins on "relevant buildings" but never explains how a building name extracted from the NLP JSON maps to a coordinate on the map. UW Bothell has a small campus, but this geocoding step — matching strings like "UW1" to lat/lng — is non-trivial and failure-prone. It's a technical gap the grader may notice.

3. FERPA compliance is stated as a goal, not a plan. The proposal says the project "will be reviewed against UW's FERPA guidelines prior to public deployment," but this is deferred and vague. Given that the system logs student interaction data tied (even hashed) to NetIDs and academic majors, a brief concrete description of *how* the review will happen — or confirmation that the data handling already satisfies FERPA — would be more credible than a forward-looking promise.